Operations, Type Coercion, and Safety

PHP has a wealth of built-in features designed to make text processing a breeze, and this includes use cases spanning multiple geographical regions. Unicode and UTF-8 are the open standards that the internet uses to enable platforms and systems to interoperate with as little friction as possible. In addition, there are common data interchange formats such as JSON and XML, which are de facto standards on the web today; PHP offers deep support for both (and more). This chapter examines all of this in detail, but now we'll turn our attention to the low-level tools that exist for manipulating strings.

Operators and Type Coercion

The previous section demonstrated a common string operation: concatenation. That's pretty much the only string-specific operation in the language, but comparison operators will also work with strings.

```
var_dump("Stardate " . "43989.1");
var_dump("abc" > "xyz");
var_dump("123" == 123);
var_dump("123" === 123);
```

```
Output:
string(16) "Stardate 43989.1"
bool(false)
bool(true)
bool(false)
```

Bear in mind that when comparing two strings, all that counts are the byte values; the comparison `"abc" > "xyz"` simply compares three bytes to three bytes. The alphanumeric collation that we interpret (as humans that can read the Latin alphabet) is just how ASCII was designed. Remember, this all falls apart once we step outside of the ASCII range (as we saw with the Polish alphabet in the previous section).

The numerical comparisons in the final two lines are a little different. `"123" == 123` is not a type-strict comparison, so coercion will happen: casting either of those values so they match types results in either two identical strings or two identical integers. The final comparison is type-strict, so it can never be true (a string is not an integer). Chapter 7, "Numbers," contains many examples of casting or coercing strings to numbers